

小实验：训练数据输入策略设计

随机重复采样 vs 按box网格输入 · SpUNet on S3DIS

策略A (Pilot A)：现状复刻，随机球形裁剪重复采样

策略B (Pilot B)：确定性立方体网格分块，遍历覆盖整场景

生成日期：2026 年 06 月 24 日

指标	baseline (500ep)	Pilot A (120ep)	Pilot B (120ep)	B - A
mIoU	64.87%	56.82%	58.93%	+2.11
mAcc	71.29%	63.31%	65.96%	+2.65
allAcc	89.05%	86.17%	85.32%	-0.85

结论速览：Pilot B 整体 mIoU 比 Pilot A 高 2.1 个百分点，方向上支持「按box均匀覆盖能缓解随机采样的密度偏差」这一假设，但并非全面领先——door 类别明显下降，详见第四、五节分析。

一、实验背景与理论原理

进入crop_size几何特征大实验之前，需要先确定网络的输入喂入方式。当前baseline用的「重复采样」机制（SphereCrop, mode=random）：每次访问场景时，从场景所有点中按点数均匀随机选一个中心点，取最近的point_max个点作为一个crop。

这个机制存在系统性的密度偏差：高密度区域（墙面、地板）被选为中心的概率远高于低密度区域（门、柱子、黑板等小物体），且point_max是固定点数而非固定半径——高密度区域对应的crop物理半径更小、更聚焦；低密度区域要凑够point_max个点，半径被迫拉大，混入周边其他类别的点，进一步稀释小物体在crop里的占比。两个效应叠加，小物体类别系统性地获得更少的学习机会。

策略B（GridBoxCrop）改为：把场景预先按固定边长的立方体网格做确定性分块，遍历覆盖整场景，每个box被访问的概率不再依赖局局部点密度，理论上能让稀疏小物体和密集大平面获得均衡的学习机会。代价是box之间存在硬边界，可能切断跨box的连续结构（如细长的门）——这正是第五节分析里发现的实际代价。

二、实验设计

项目	策略A · Pilot A	策略B · Pilot B
裁剪方式	SphereCrop(mode=random)，随机球形裁剪	GridBoxCrop(crop_size=6.0m, jitter=0.3m)
位置选取	每次访问随机选中心点	预先网格分块，box_min_points=200过滤
loop / 样本量	loop=1，204个训练场景	loop=1，323个box（Area_1/2/3/4/6）
每epoch crop数	约204	约323（已知不完全相等，见下方说明）
模型	SpUNet-v1m1（两策略相同结构）	同左
epoch数	120	120（同左，保证逐epoch可比）
其余增强pipelineRandomRotate/Scale/Jitter/Elastic/Chromatic 等，参数与B完全相同		同左
测试协议	Area_5全场景，10x TTA（5尺度×2翻转）	同左

注：两策略每epoch的crop数不完全相等（204 vs 323），这是已知妥协——Pointcept框架内部用 loop = epoch // eval_epoch 重新计算loop，强行调大A的loop会打乱逐epoch可比性，故保留loop=1，接受这个折中。

三、整体结果

Area_5全场景测试（68个房间，10x TTA），三组结果对比：

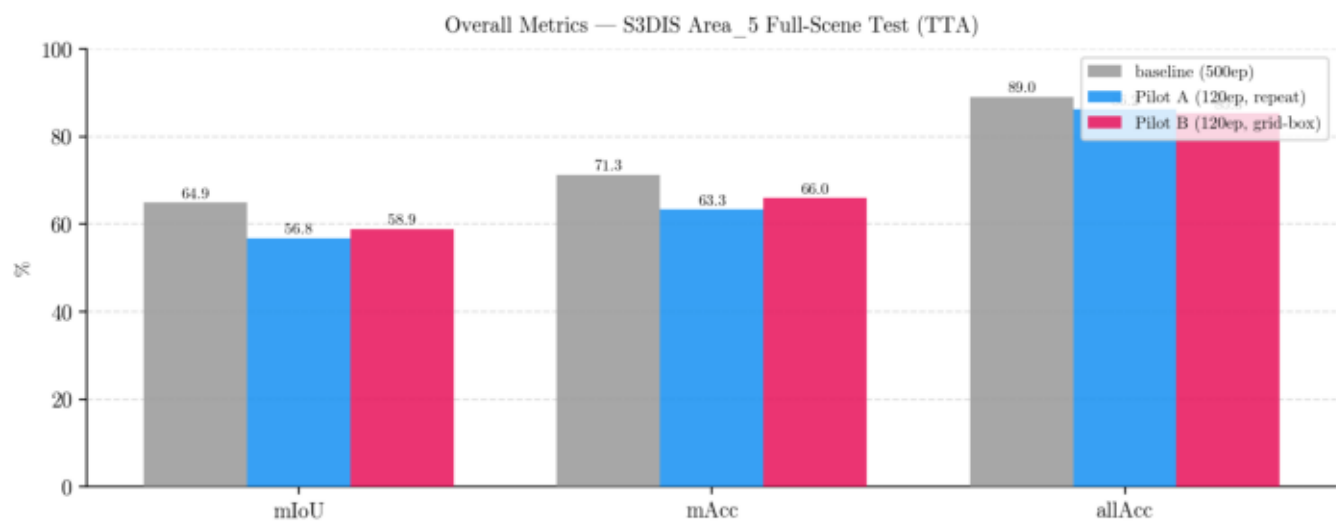


图1：整体 mIoU / mAcc / allAcc 对比

实验	训练耗时	是否崩溃
baseline (500ep)	约2小时8分	无
Pilot A (120ep)	约40分钟（修复数据后）	无（修复前崩溃过4次OOM）
Pilot B (120ep)	约1小时（含1次崩溃恢复）	epoch79崩溃1次，已修复并恢复

四、各类别 IoU 详细对比

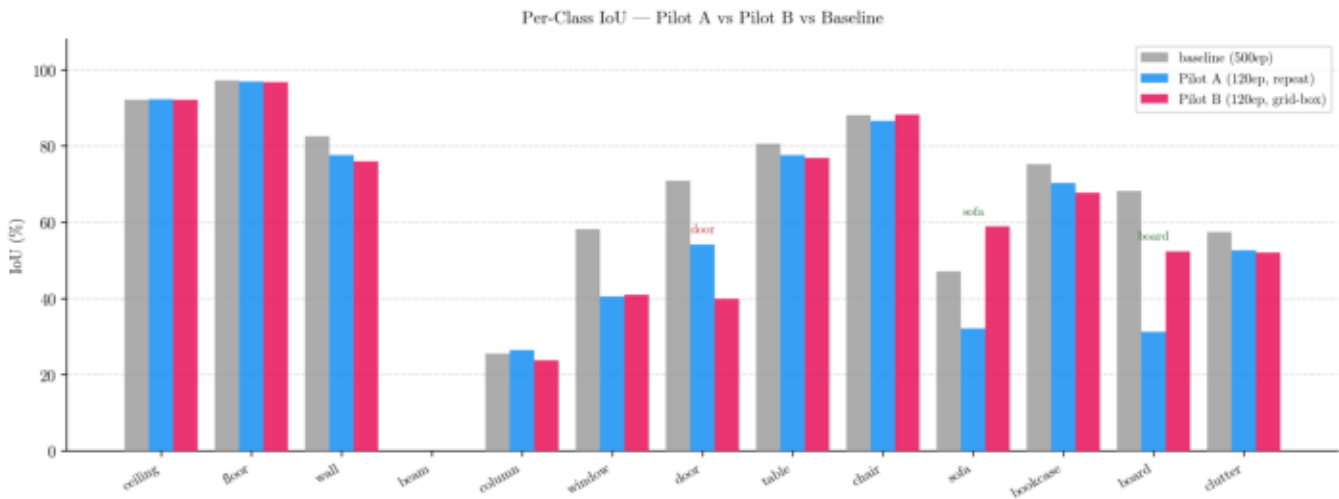


图2: 13个类别 IoU 对比 (标注差异最大的三个类别)

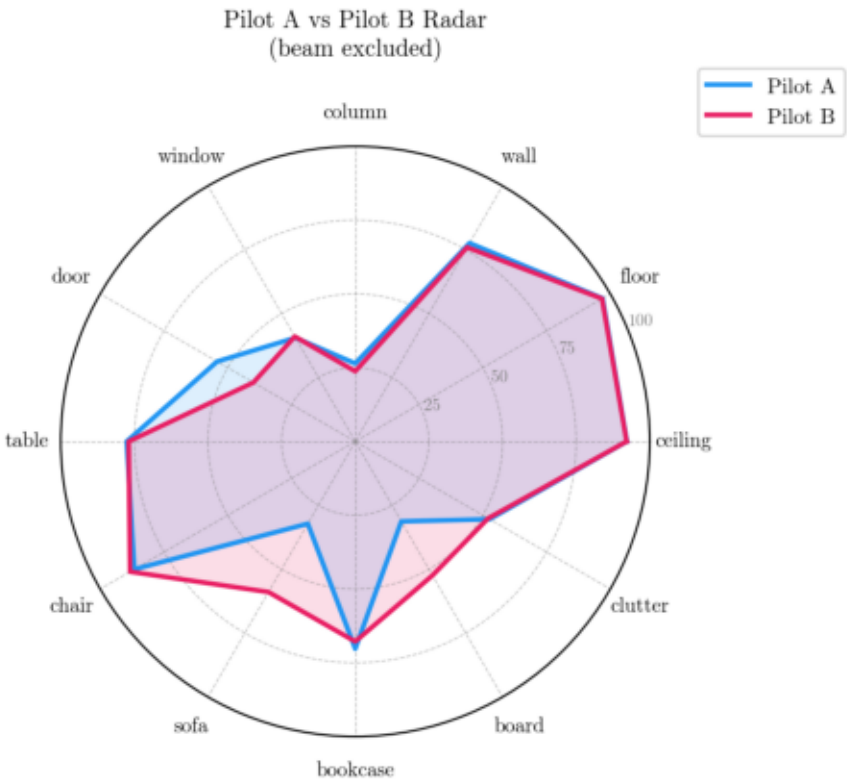


图3: Pilot A vs Pilot B 雷达图 (剔除beam)

四、各类别 IoU 详细对比（续）

类别	baseline	Pilot A	Pilot B	B-A
天花板 (ceiling)	92.21	92.24	92.15	-0.09
地板 (floor)	97.23	96.98	96.76	-0.22
墙壁 (wall)	82.57	77.60	75.96	-1.64
横梁 (beam)	0.00	0.00	0.00	+0.00
柱子 (column)	25.55	26.43	23.78	-2.65
窗户 (window)	58.14	40.57	41.05	+0.48
门 (door)	70.86	54.17	39.94	-14.23
桌子 (table)	80.70	77.60	76.93	-0.67
椅子 (chair)	88.15	86.63	88.28	+1.65
沙发 (sofa)	47.08	32.20	58.93	+26.73
书架 (bookcase)	75.24	70.26	67.80	-2.46
黑板 (board)	68.16	31.29	52.43	+21.14
杂物 (clutter)	57.45	52.70	52.10	-0.60

橙色底色行：B-A差值绝对值 ≥10个百分点的类别（door、sofa、board）。

五、结果分析与讨论

5.1 整体结论：方向上支持假设，但不是压倒性优势

Pilot B整体mIoU比Pilot A高2.1个百分点，方向上支持理论假设，但幅度不大，且各类别表现分化明显。

5.2 sofa、board大幅提升（+26.7、+21.1个点）——符合理论预期

这两个类别都是稀疏分布、单体占用空间不大的家具类，正是理论分析中最容易被密度偏差压制的对象。策略B均匀遍历后，这些物体所在box的访问频率不再受周围点密度影响，学习机会显著增加。这是本次实验里最支持理论假设的证据。

5.3 door明显下降（-14.2个点）——可能与box硬边界切割门有关

门是细长结构，嵌在墙体窄缝里，6m立方体网格的边界可能正好把门切碎，破坏几何完整性；而球形裁剪以随机点为中心，至少有一定概率把整扇门完整收进某次crop。box网格是格点对齐、确定性的，如果门正好跨在box分界线上，每个epoch都会被同样的方式切开，没有随机性来‘碰运气’凑出完整样本。这是策略B暴露出的一个真实代价，建议后续对门窗类细长结构做特殊分块处理。

5.4 column/wall/bookcase小幅下降、beam两边均为0

Pilot B每epoch crop数（323）比Pilot A（204）多约58%，但column/wall/bookcase反而小幅下降，说明样本量差异不是主导因素，更可能是box边界几何完整性的影响，程度比door轻。beam类别在三组实验里都是0 IoU，是数据集本身的限制（样本极少），与采样策略对比无关。

5.5 训练成本：数据集事故 vs 修复后的对比



图4：每epoch的worker重启等待耗时，数据集体积化修复前后对比

实验过程中发现数据集被意外还原成未体积化的原始版本（点数暴涨~20倍），导致Pilot A训练耗时一度暴涨13倍（128秒/epoch），测试阶段反复OOM崩溃。修复后（重新体积化并持久化存储），两个pilot的训练速度都恢复到与baseline一致的水平（~15秒/epoch），显存消耗稳定在容器50GB上限的安全范围内（实测峰值约20GB），验证了「显存消耗与裁剪策略本身无关，只与point_max有关」这一设计前提。详细排查过程见《环境与配置基线_避坑指南》。

六、结论

1. 按box输入（策略B）整体mIoU优于随机重复采样（策略A）约2.1个点，方向上支持「随机采样存在密度偏差、均匀覆盖能缓解」的理论假设。
2. 证据最强的支持点在于sofa、board两个稀疏小物体类别的大幅提升（+26.7、+21.1点），与理论分析高度吻合。
3. 但door类别明显下降（-14.2点），暴露了box策略「确定性硬边界可能切碎细长结构」的潜在代价，这是理论分析里没有预先考虑到的新发现。
4. 两策略收敛速度、训练成本（时间、显存）基本相当，不存在哪种策略明显更快/更省资源的优势。

七、局限性与后续工作建议

1. 本次只跑了单次种子，建议换种子重跑确认2.1个点的差距在统计上是否稳定（crop数204 vs 323不完全相等，存在一定混杂因素）。
2. door类别下降值得专门深挖：检查测试集里door样本被box边界切割的实际比例，验证「边界切割」假设。
3. 建议进入crop_size大实验前，先确认box策略是否需要针对门窗类结构做特殊处理，否则后续18组实验都会继承这个已知局限。
4. exp/baseline_ptv3需要用修复后的数据集重新训练，目前结果是在错误数据上跑的，不可信，不建议用于PTv3 vs SpUNet的横向对比。

完整文字版报告（含理论推导细节、数据集事故完整时间线）见 docs/小实验_完整总结报告.txt；排查过程见 docs/环境与配置基线_避坑指南.txt。